

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**

Introdução à Ciência de Dados, Ferramentas e Jargões da Área

Termos técnicos na Ciência de Dados

Aula 3

Código da aula: [DADOS]ANO1C1B1S2A3

Exposição



Objetivos da Aula

- Conhecer os termos técnicos utilizados na área de Ciência de Dados e Inteligência Artificial.



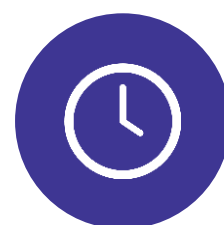
Competências da Unidade (Técnicas e Socioemocionais)

- Comunicar ideias de forma clara e assertiva, sabendo ouvir ativamente, visando à construção de relacionamentos sólidos.
- Colaborar efetivamente com outros profissionais, como cientistas de dados e engenheiros de dados.
- Trabalhar em equipe, compartilhando conhecimentos, contribuindo com ideias e colaborando para alcançar objetivos comuns.



Recursos Didáticos

- Recurso audiovisual para exibição de vídeos e imagens.
- Acesso ao laboratório de informática e/ou internet.



Duração da Aula

50 minutos

Cronograma da aula de hoje

Termos técnicos na Ciência de Dados

- ✓ Introdução e revisão de vocabulário técnico.
- ✓ Exemplos de seus significados.
- ✓ Atividade.

Exposição

Momento de revisão!

Em grupos e com a ajuda do(a) professor(a), faça uma revisão da aula passada.

- ✓ Quais foram os principais tópicos vistos?
- ✓ Você se lembra dos exemplos dados?
- ✓ Fale sobre o que aprendeu para os seus colegas.

Exposição

Ferramentas e práticas: Python para Ciência de Dados

Python é uma linguagem de programação de propósito geral que se tornou uma das mais populares linguagens para a Ciência de Dados. Sua simplicidade e legibilidade tornam a programação em Python mais fácil do que em muitas outras linguagens.

Ferramentas e práticas: Python para Ciência de Dados

Existem várias **bibliotecas** Python que são úteis para a Ciência de Dados. **NumPy** e **pandas** são usadas para manipulação de dados. **Matplotlib** e **Seaborn** são usadas para visualização de dados. **Scikit-learn** é uma biblioteca abrangente para o aprendizado de máquina, que inclui muitos algoritmos populares. **TensorFlow** e **PyTorch** são bibliotecas de aprendizado profundo, usadas para construir e treinar redes neurais.



Python/Dicionário do programador



© Getty Images

Exposição



Fonte: <https://www.youtube.com/watch?v=uOgDaIrlqjE>.

Exposição

Ferramentas e práticas: NumPy (*Numerical Python*)

NumPy é uma biblioteca Python que fornece suporte para **grandes matrizes e matrizes multidimensionais**, com uma grande coleção de funções matemáticas de alto nível para operar nesses arrays.

Exemplo de uso da NumPy para a criação de uma matriz:

```
import numpy as np

# Criar um array 2D (uma matriz)
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Calcular a média de todos os elementos da matriz
mean = np.mean(matrix)
```

Elaborado especialmente para o curso.



Reprodução – NumPy. Disponível em:
<https://numpy.org/>. Acesso em: 10 jan. 2024.

Exposição



Reprodução – pandas. Disponível em:
<https://pandas.pydata.org/>. Acesso em: 10 jan. 2024.

Ferramentas e práticas: pandas

Pandas é uma biblioteca **Python** que fornece estruturas de **dados de alto desempenho e ferramentas de análise de dados**. Os principais tipos de dados que o pandas manipula são as Series (arrays unidimensionais com rótulos) e os *DataFrames* (tabelas bidimensionais onde cada coluna é uma série). Exemplo de uso do pandas para a criação de uma tabela:

```
import pandas as pd

# Ler dados de um arquivo CSV
df = pd.read_csv('data.csv')

# Ver as primeiras linhas dos dados
print(df.head())

# Calcular a média de uma coluna
mean = df['Column1'].mean()
```

Elaborado especialmente para o curso.

Exposição



Reprodução – Matplotlib. Disponível em:
<https://matplotlib.org/>. Acesso em: 10 jan. 2024.

Ferramentas e práticas: Matplotlib

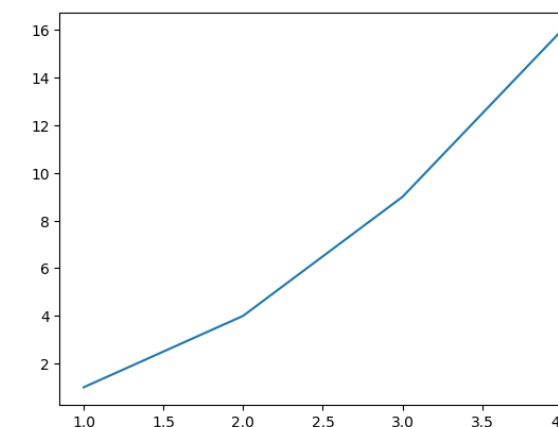
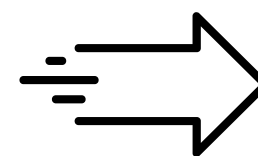
Matplotlib é uma biblioteca de plotagem para a linguagem de programação Python e sua extensão matemática NumPy. Ela fornece uma maneira de visualizar dados em Python.

Exemplo de uso do Matplotlib para a criação de um gráfico:

```
import matplotlib.pyplot as plt

# Plotar um gráfico de linha
plt.plot([1, 2, 3, 4], [1, 4, 9, 16])
plt.show()
```

Elaborado especialmente para o curso.



Elaborado especialmente para o curso.

Exposição

Ferramentas e práticas: Seaborn

Seaborn é uma biblioteca de visualização de dados Python baseada no Matplotlib. Ela fornece uma interface de alto nível para desenhar gráficos estatísticos atraentes.

Exemplo de uso do Seaborn para a criação de um gráfico:

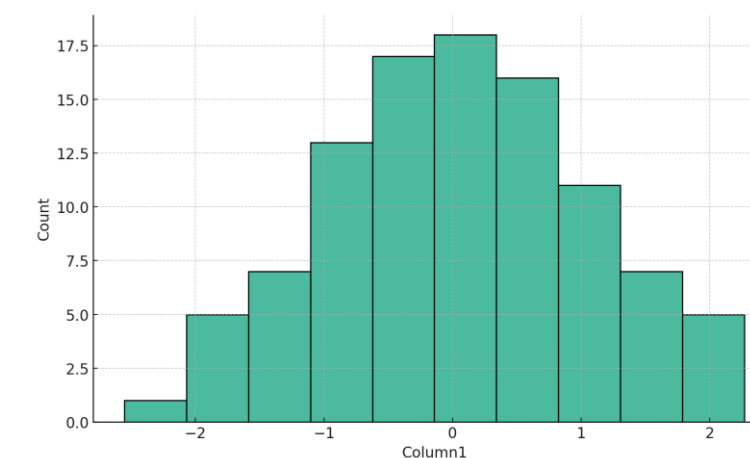
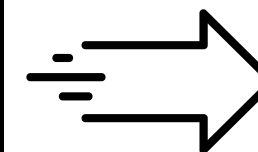


Reprodução – Seaborn. Disponível em:
<https://seaborn.pydata.org/>. Acesso em: 10 jan. 2024.

```
import seaborn as sns

# Visualizar um histograma
sns.histplot(data=df, x="Column1", bins=10)
```

Elaborado especialmente para o curso.



Elaborado especialmente para o curso.

Exposição

Ferramentas e práticas: Scikit-learn

Scikit-learn é uma biblioteca de aprendizado de máquina em Python. Ela inclui vários algoritmos de classificação, regressão e clustering, incluindo máquinas de vetores de suporte, florestas aleatórias, *gradient boosting*, *k-means* e DBSCAN.

Exemplo de uso do Scikit-learn
para criar um classificador
Randon Forest:



Reprodução – Scikit-Learn. Disponível em:
<https://scikit-learn.org/stable/index.html>.
Acesso em: 10 jan. 2024.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

# Dividir os dados em conjunto de treinamento e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Criar um classificador de floresta aleatória
clf = RandomForestClassifier()

# Treinar o modelo
clf.fit(X_train, y_train)

# Fazer previsões no conjunto de teste
predictions = clf.predict(X_test)
```

Elaborado especialmente para o curso.

Exposição



Reprodução – PyTorch. Disponível em:
<https://pytorch.org/>. Acesso em: 10 jan. 2024.



Reprodução – TensorFlow. Disponível em:
<https://www.tensorflow.org/?hl=pt-br>. Acesso em: 10 jan. 2024.

Ferramentas e práticas: TensorFlow e PyTorch

TensorFlow e PyTorch são bibliotecas de aprendizado profundo que permitem a construção de redes neurais complexas. Ambas têm suporte para GPUs, permitindo o rápido treinamento de grandes modelos.

Exemplo de uso do TensorFlow para criar uma rede neural artificial:

```
import tensorflow as tf

# Criar um modelo sequencial
model = tf.keras.models.Sequential([
    tf.keras.layers.Dense(10, activation='relu', input_shape=(32,)),
    tf.keras.layers.Dense(2, activation='softmax')
])

# Compilar o modelo
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy')

# Treinar o modelo
model.fit(X_train, y_train, epochs=5)
```

Elaborado especialmente para o curso.

Ferramentas e práticas: pré-processamento de dados

O pré-processamento de dados é um passo crítico na *pipeline* de ciência de dados. Envolve limpar e transformar dados brutos em um formato que **os algoritmos de aprendizado de máquina podem usar**. Isso pode incluir a remoção de outliers, a manipulação de dados faltantes, a normalização de características e a codificação de variáveis categóricas.



Exemplo

Muitos algoritmos de aprendizado de máquina requerem que todos os recursos sejam numéricos. Se você tiver uma característica categórica (como "cor dos olhos"), terá de codificar essa característica como números (por exemplo, "azul" = 1, "verde" = 2, "castanho" = 3).

Ferramentas e práticas: treinamento e avaliação de modelos

O treinamento de um modelo é o processo de **ajustar os parâmetros de um algoritmo de aprendizado de máquina com base nos dados** de treinamento para que o modelo possa fazer previsões precisas.

Por exemplo: Em primeiro lugar, importamos a biblioteca e criamos uma instância do modelo de Regressão Linear. Em seguida, usamos o método fit para treinar o modelo nos dados de treinamento:

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Suponha que temos X como nossos recursos e y como nossos alvos
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Criação do modelo
model = LinearRegression()

# Treinamento do modelo
model.fit(X_train, y_train)
```

Elaborado especialmente para o curso,

Ferramentas e práticas: avaliação de modelos

A avaliação do modelo é o processo de verificar o desempenho de um modelo treinado. Isso geralmente é feito usando um conjunto de dados de teste que não foi visto durante o treinamento.

A avaliação do modelo pode ser feita de diferentes maneiras, dependendo do tipo de tarefa de aprendizado de máquina. Para problemas de regressão, as métricas comuns incluem erro quadrático médio (MSE), erro absoluto médio (MAE) e R quadrado. Para problemas de classificação, as métricas comuns incluem precisão, recall, F1-score e área sob a curva ROC (AUC-ROC).

Ferramentas e práticas: avaliação de modelos

No exemplo de Regressão Linear abaixo, podemos avaliar o modelo calculando o R quadrado nos dados de teste:

```
# Avaliação do modelo
score = model.score(X_test, y_test)
print(f"R^2 score: {score}")
```

Elaborado especialmente para o curso.

```
from sklearn.metrics import mean_squared_error

# Fazendo previsões nos dados de teste
y_pred = model.predict(X_test)

# Calculando MSE
mse = mean_squared_error(y_test, y_pred)
print(f"MSE: {mse}")
```

Elaborado especialmente para o curso.

Ferramentas e práticas: trabalho em equipe e colaboração

A Ciência de Dados frequentemente é um esforço de equipe, e a **colaboração eficaz é crucial para o sucesso de um projeto**. Isso pode envolver a comunicação clara de ideias, o compartilhamento de **código e dados de maneira eficiente e a organização do trabalho de maneira que todos na equipe entendam suas responsabilidades**.

Ferramentas como o **Git** e o **GitHub** geralmente são usadas para o controle de versão e colaboração em projetos de Ciência de Dados. O Jupyter Notebooks é uma ferramenta útil para compartilhar análises de dados e resultados em um formato legível que combina código, texto e visualizações.

Ferramentas e práticas: trabalho em equipe e colaboração

Exemplo: Uma equipe de cientistas de dados pode usar o GitHub para colaborar em um projeto. Cada um deles pode trabalhar em diferentes partes do projeto, em seus próprios ramos, e depois mesclar suas mudanças de volta ao ramo principal quando estiverem prontos. Eles podem usar pull requests para revisar e discutir mudanças antes de elas serem incorporadas ao projeto.



O que são Git e GitHub?



© Getty Images

Exposição



Fonte: https://www.youtube.com/watch?v=P4BNi_yPehc.

Vamos
fazer uma
atividade

Vamos elaborar um gráfico?

Represente o gráfico gerado pelo trecho de código abaixo:

```
1 import matplotlib.pyplot as plt
```

```
1 plt.plot([1, 2, 3, 4], [1, 16, 9, 10])
```

```
2 plt.show()
```

Elaborado especialmente para o curso.

Esse gráfico deve ser enviado conforme orientação do professor



© Getty Images

O que nós
aprendemos
hoje?

Hoje desenvolvemos:

- 1** Capacidade de **compreender o vocabulário técnico da área de Ciência de Dados.**
- 2** Conhecimento sobre **quais são as principais bibliotecas e funções do Python.**
- 3** Aprendizado sobre **a diferença do Git e GitHub** por meio de um vídeo.

Saiba mais

Os algoritmos precisam de medições para aferir sua eficiência e como eles executam seus cálculos. Portanto, é fundamental desenvolver e aplicar métricas que possibilitem essa análise.

DATA ARTE. Métricas de Avaliação em Ciência de Dados – Parte 1. Disponível em: https://www.youtube.com/watch?v=2eNz4T_Rnis. Acesso em: 9 jan. 2024.

DATA ARTE. Métricas de Avaliação em Ciência de Dados – Parte 2. Disponível em: <https://www.youtube.com/watch?v=Im9GxJSdofM>. Acesso em: 9 jan. 2024.

A principal etapa do processo de análise de dados é a preparação dos dados. Assista a um vídeo sobre a importância dessa etapa:

UNIVESP. Introdução à Ciência de Dados – Preparação e Pré-Processamento de Dados – Parte 1. Disponível em: <https://www.youtube.com/watch?v=pbksL5cpeSk>. Acesso em: 9 jan. 2024.

Referências da aula

ALURA. **O que são Git e GitHub?** #HipstersPontoTube. Disponível em: https://www.youtube.com/watch?v=P4BNi_yPehc. Acesso em: 9 jan. 2024.

AMARAL, Fernando. **Introdução à Ciência de Dados: mineração de dados e *Big Data***. Rio de Janeiro: Alta Books, 2018.

CÓDIGO FONTE TV. Python // **Dicionário do programador**. Disponível em: <https://www.youtube.com/watch?v=uOgDaIrlqjE>. Acesso em: 9 jan. 2024.

Identidade visual: imagens © Getty Images.

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**